

Project proposal - Control-Flow Integrity

Filip Redhammar, Jack Lindberg, Carl Tydén
DD249U - System security project

Background/Introduction

Our objective is to implement security measures in a small kernel for educational purposes. Our project idea is to utilize the capability-based separation kernel project called s3k provided and maintained by the Royal institute of technology. Our proposal is to implement a control-flow integrity feature that partially protects and mitigates certain types of binary exploitation.

Problem Description

The security problem we would like to address is control-flow hijacking through corruption of function return addresses. Any vulnerability that allows an attacker to overwrite a function's return address on the stack can potentially redirect a program's execution if no relevant mitigation techniques are present. Vulnerability classes where this exploit technique can be utilized include stack-based buffer overflows.

A shadow stack attempts to mitigate this exploitation technique by keeping a separate, protected stack where copies of the original return addresses are stored. When a function returns, if the return address does not match the corresponding value in the shadow stack, the program can respond to the attack, usually by raising an exception or terminating execution.

Control-flow integrity

Control flow integrity is implemented in order to determine if the program runs as intended by the developer. This is primarily done by validating function-calls, jumps or return addresses. This feature can be implemented in different ways.

forward-edge

The validation occurs before the function is called, and the called address is compared against a predefined set of possible addresses. This predefined address lists are parsed before runtime.

backwards-edge

Backwards-edge CFI implements functions to validate the return address of function calls performed during runtime, it saves the return address as it enters the function and compares it with the return address at end of the function to determine if it has been tampered with during the run.

shadow stack

This is a implementation closely related to the above feature. A shadow stack is a separate stack that stores the saved return addresses. This makes it possible to compare the return address recursively using a LIFO stack.

Implementation

We are leaning against using a backwards-edge implementation, but there are still some concerns regarding the implementation of choice. This solution could be accomplished in various ways.

At compile time:

- Compilation flags are inserted by the compiler. This enables the program to communicate externally with our implemented monitor.

Issues:

- This implementation is conventionally implemented using hardware. Software-based implementation could lead to unnecessary overhead.
- Handling recursive calls could lead to overwriting the shadow-stack buffer.
- A malicious user could be able to overwrite the functions that communicates the return address with the monitor.

Post compile time:

- Parsing of the addresses are performed during runtime, this requires that the symbol-table is extracted or somehow that the executable runs monitored in a sandboxed environment with limited capabilities.

Issues:

- How to properly hook all returns during runtime.
- When to properly parse it? Is it possible to run the program sandboxed or without capabilities and parse the addresses before the program runs by the user.
- How to actually read the return address from memory in runtime?
- What would happen if a malicious actor overwrites an entire function?